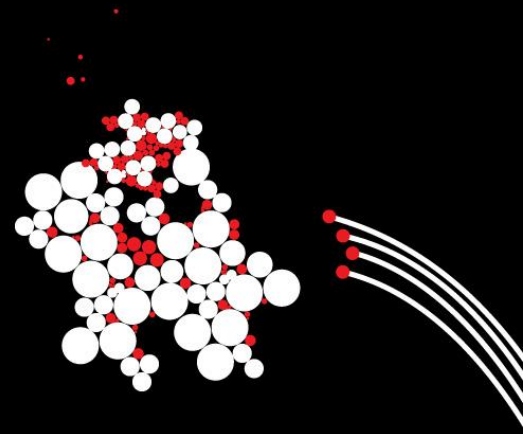


UNIVERSITY OF TWENTE.



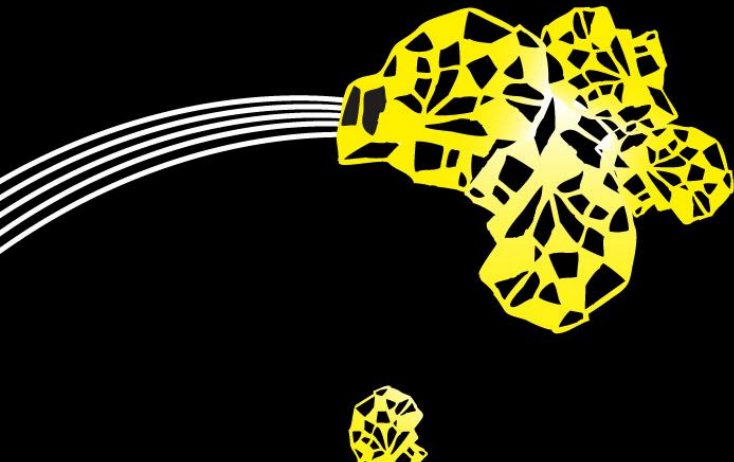
MEDICALCPS2014



IDSL: A DOMAIN SPECIFIC LANGUAGE FOR PERFORMANCE EVALUATION OF MEDICAL IMAGING SYSTEMS

14TH OF APRIL, 2014

FREEK VAN DEN BERG, ANNE REMKE, BOUDEWIJN HAVERKORT



COMMIT/ALLEGIO

Embedded Systems
Innovation
BY TNO

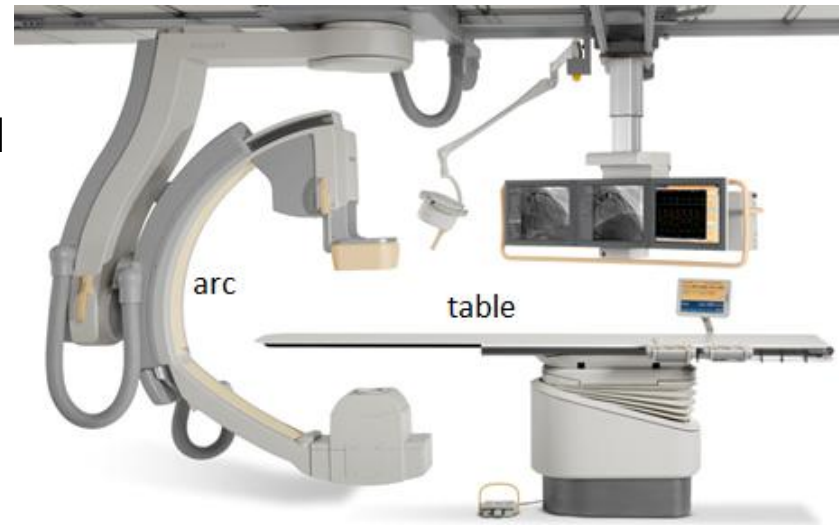
PHILIPS
sense and simplicity

OVERVIEW

- Introduction
- iDSL tool and solution chain
- iDSL language constructs
- iDSL performance evaluation
 - Way of working
 - MODES simulations
 - UPPAAL model checking
- Conclusion

INTRODUCTION

- **Medical Imaging Machines** (MIS) perform safety critical tasks
- **iXR machines** are MIS
 - dynamically record images, X-ray based
 - measurement-based design decisions
- **Earlier work**
 - a POOSL simulation approach
- **MoDeST** provides a modeling language
 - Stochastic Timed Automata (STA) based
 - analysis through PRISM, **UPPAAL** and **simulation**



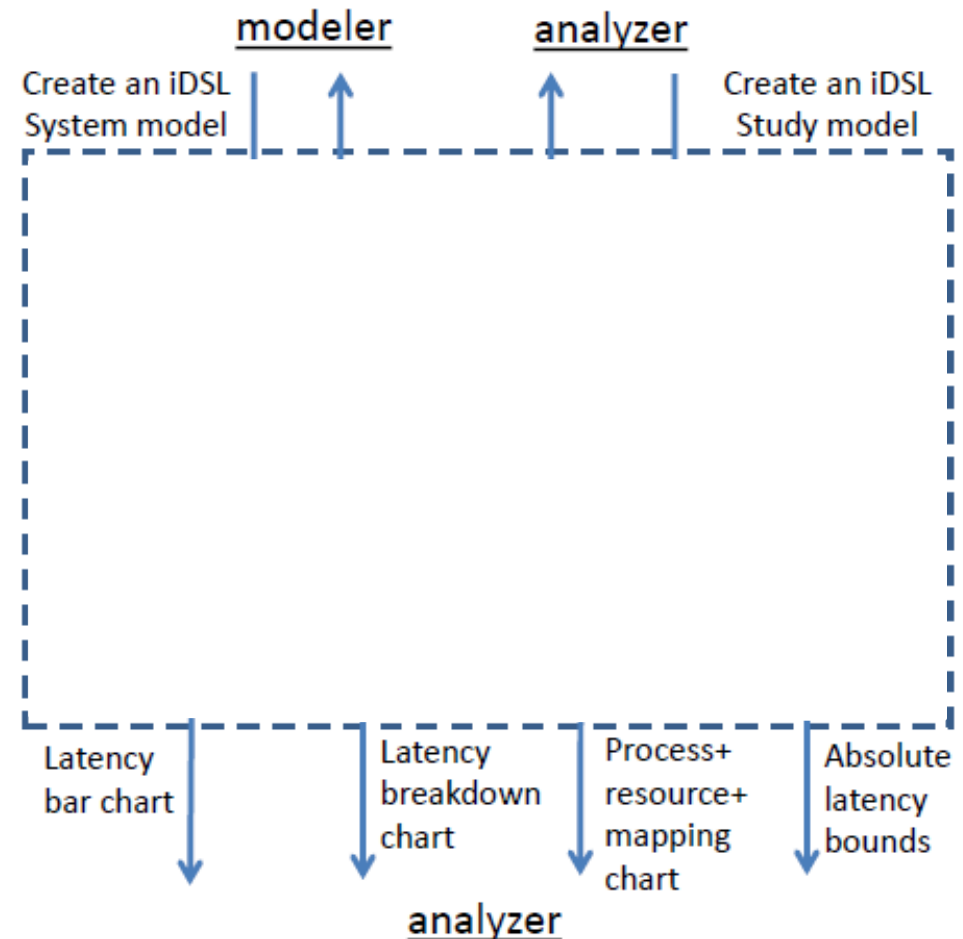
Modest Toolset

OVERVIEW

- Introduction
- **iDSL tool and solution chain**
- iDSL language constructs
- iDSL performance evaluation
 - Way of working
 - MODES simulations
 - UPPAAL model checking
- Conclusion

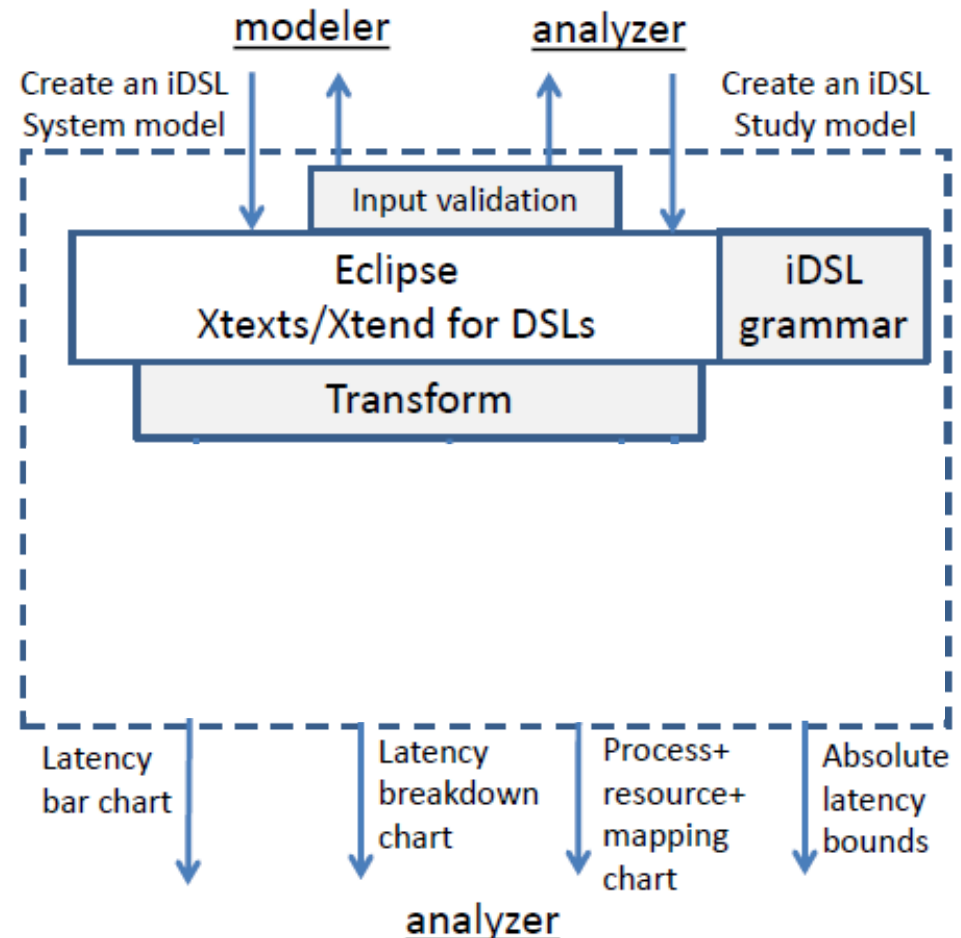
IDSL TOOL AND SOLUTION CHAIN

- **iDSL**
 - Domain specific language
 - Performance evaluation of MIS
 - Transforms to MoDeST
 - Visualization of results ()
 - Fully automated
- Why not use MoDeST directly?
 - Low level, less domain specific
 - One model does **not** fit all



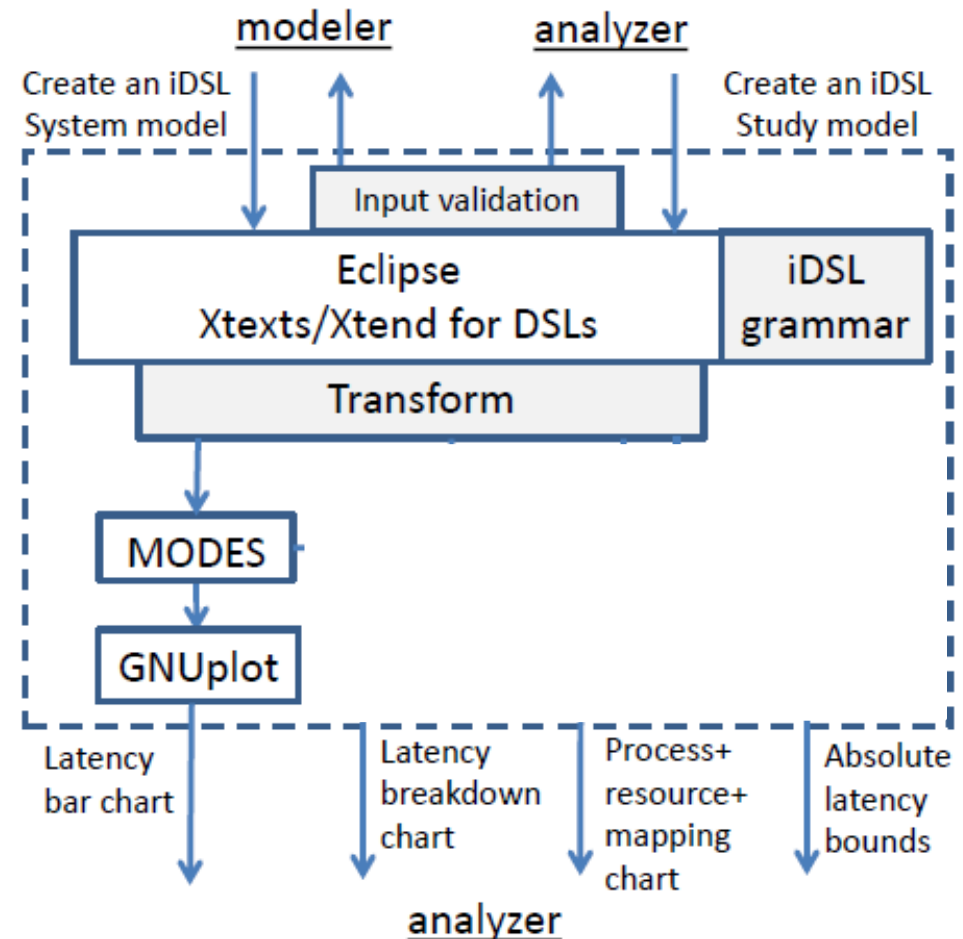
IDSL TOOL AND SOLUTION CHAIN

- **iDSL**
 - Domain specific language
 - Performance evaluation of MIS
 - Transforms to MoDeST
 - Visualization of results ()
 - Fully automated
- Why not use MoDeST directly?
 - Low level, less domain specific
 - One model does **not** fit all



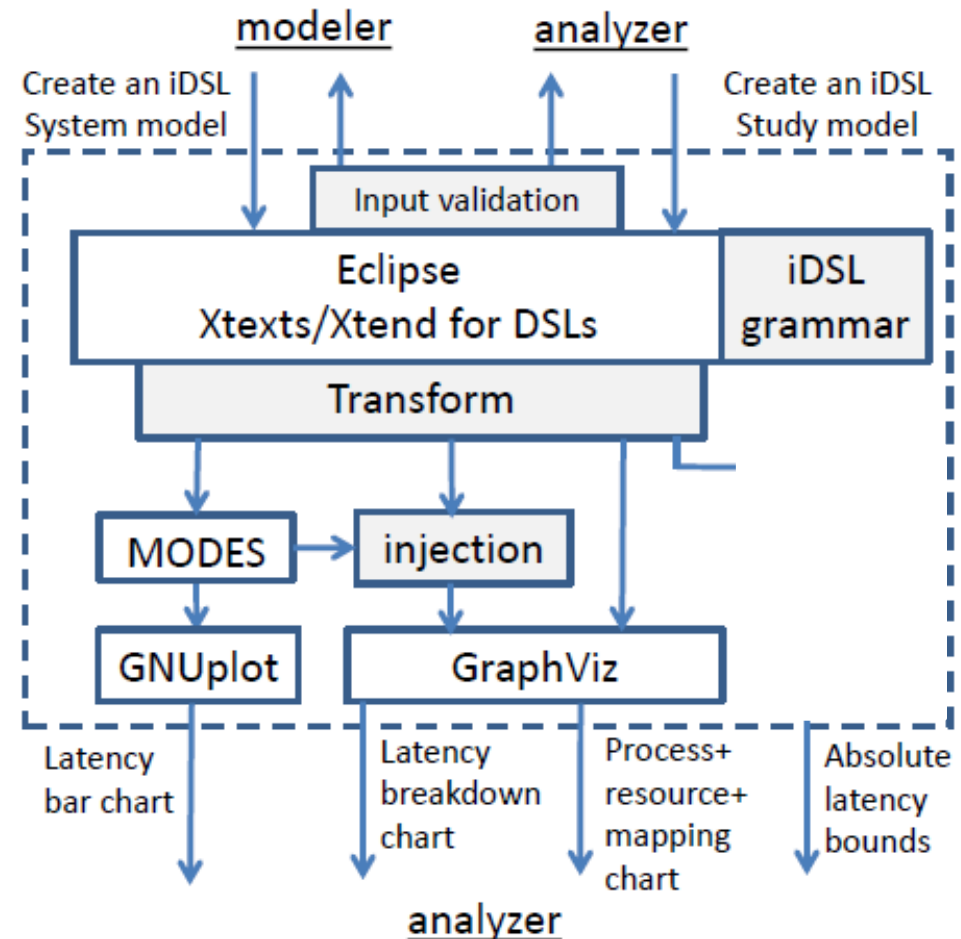
IDSL TOOL AND SOLUTION CHAIN

- **iDSL**
 - Domain specific language
 - Performance evaluation of MIS
 - Transforms to MoDeST
 - Visualization of results ()
 - Fully automated
- Why not use MoDeST directly?
 - Low level, less domain specific
 - One model does **not** fit all



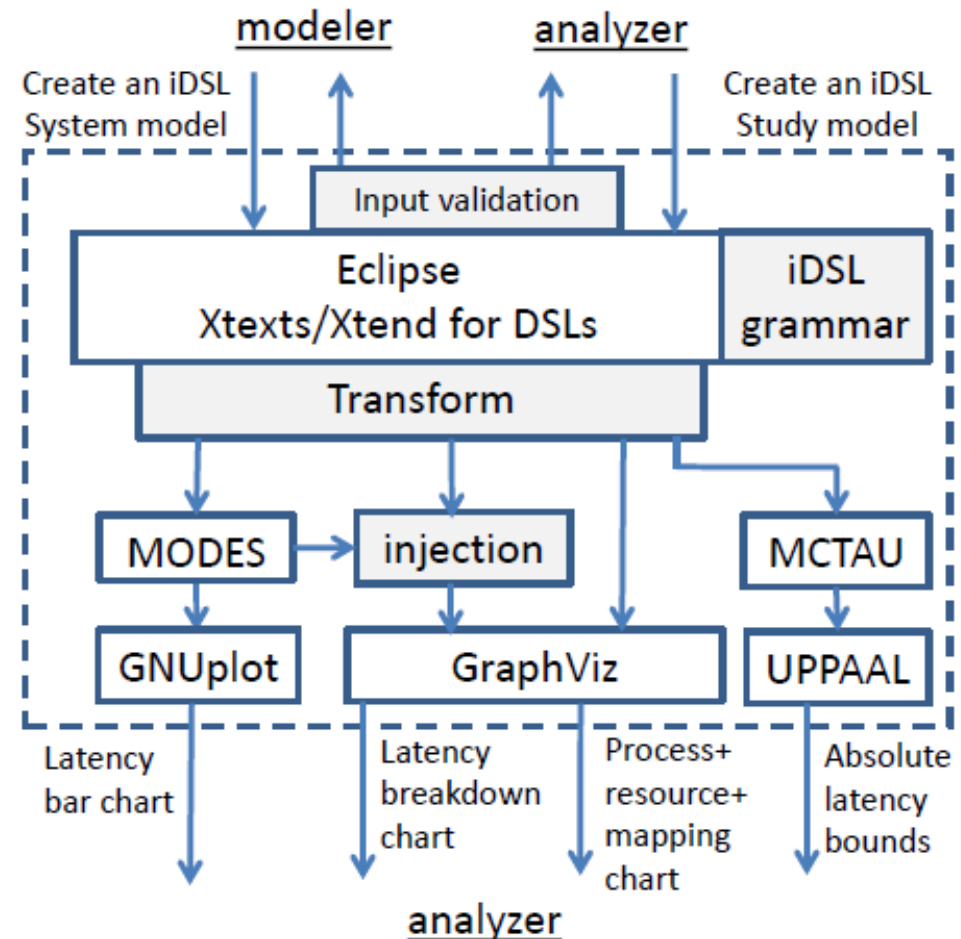
IDSL TOOL AND SOLUTION CHAIN

- **iDSL**
 - Domain specific language
 - Performance evaluation of MIS
 - Transforms to MoDeST
 - Visualization of results ()
 - Fully automated
- Why not use MoDeST directly?
 - Low level, less domain specific
 - One model does **not** fit all



IDSL TOOL AND SOLUTION CHAIN

- **iDSL**
 - Domain specific language
 - Performance evaluation of MIS
 - Transforms to MoDeST
 - Visualization of results ()
 - Fully automated
- Why not use MoDeST directly?
 - Low level, less domain specific
 - One model does **not** fit all

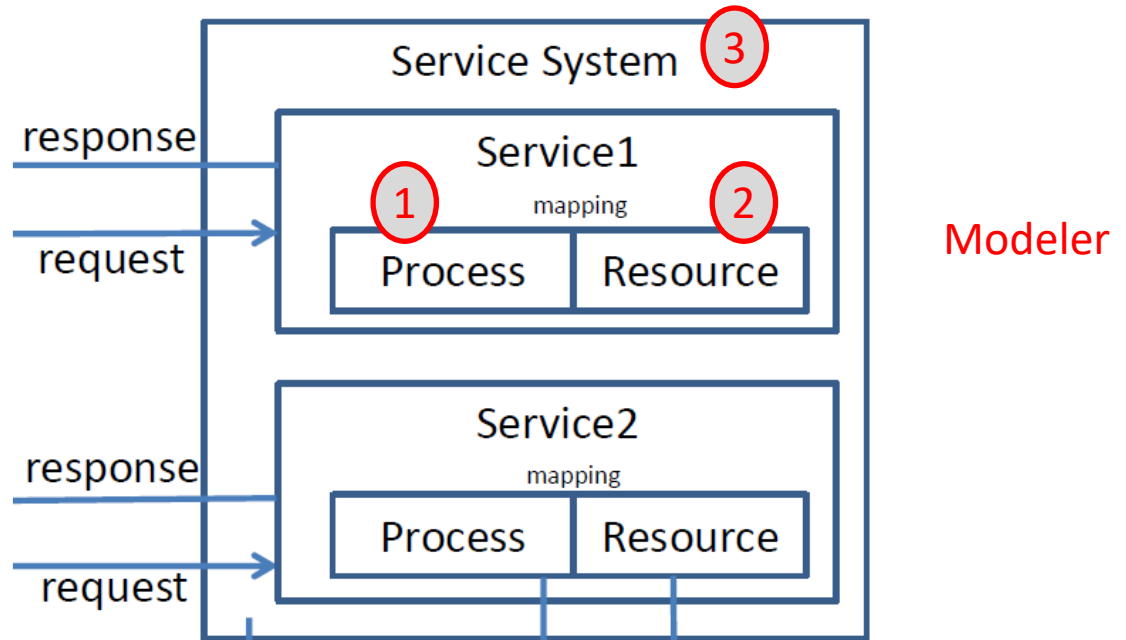


OVERVIEW

- Introduction
- iDSL tool and solution chain
- **iDSL language constructs**
- iDSL performance evaluation
 - Way of working
 - MODES simulations
 - UPPAAL model checking
- Conclusion

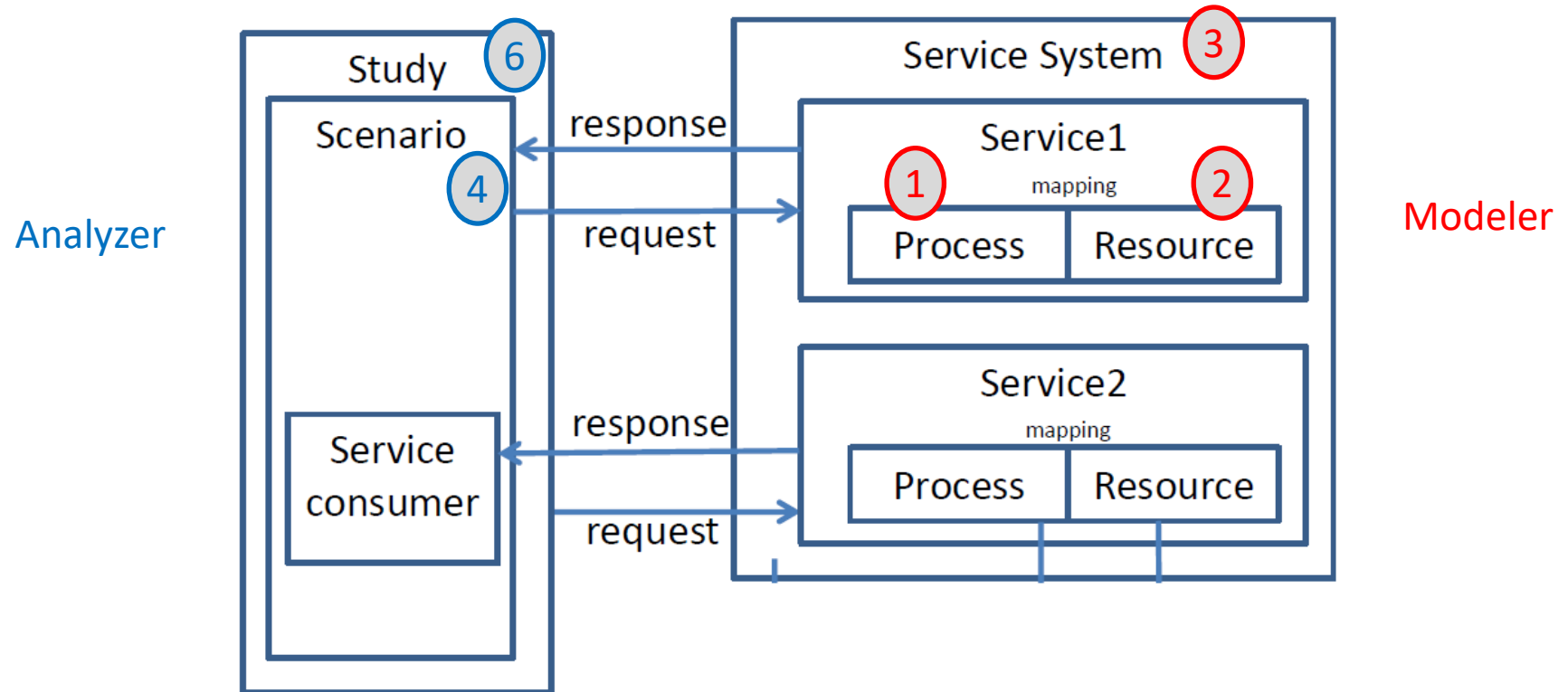
IDSL LANGUAGE CONSTRUCTS

OVERVIEW



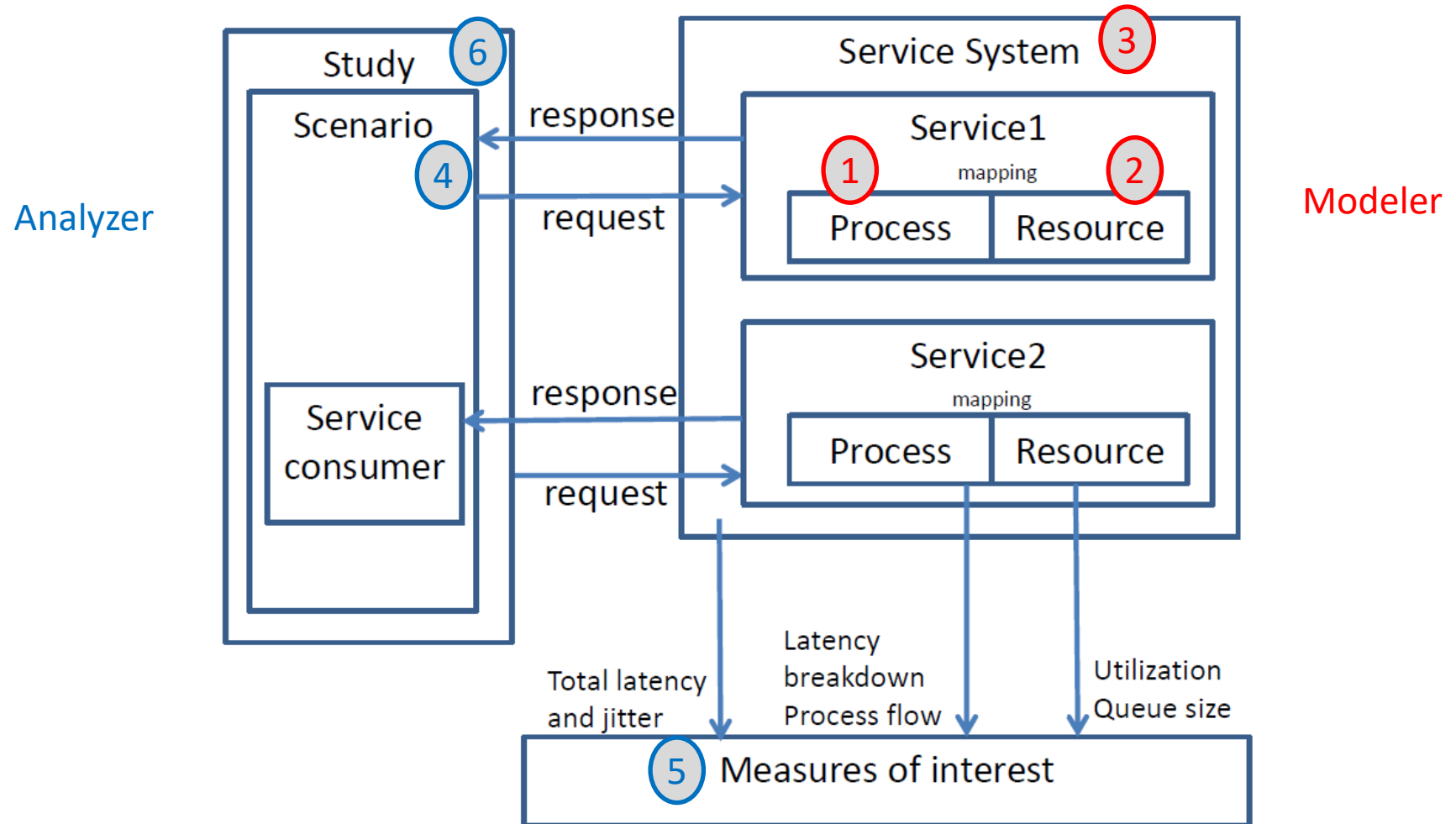
IDSL LANGUAGE CONSTRUCTS

OVERVIEW



IDSL LANGUAGE CONSTRUCTS

OVERVIEW

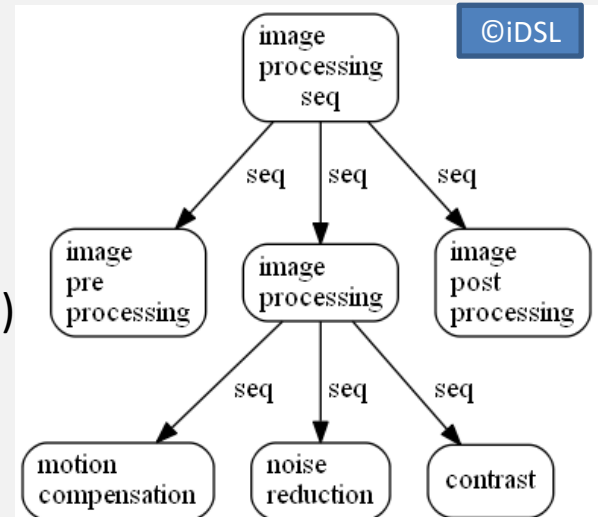


LANGUAGE CONSTRUCTS

PROCESS, RESOURCE, SYSTEM, SCENARIO, MEASURE AND STUDY

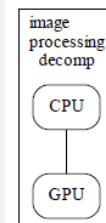
Section Process

```
ProcessModel image_processing_application seq {  
  atom image_pre_processing load 25  
  seq image_processing {  
    atom motion_compensation load 22  
    atom noise_reduction load uniform (40 70)  
    atom contrast load 67  
  }  
  atom image_post_processing load 5  
} // also supported: par, alt, palt, abstract and mutex
```



Section Resource

```
ResourceModel image_processing_PC decomp  
{ atom CPU rate 1 atom GPU rate 1 } connections { (CPU,GPU) }
```



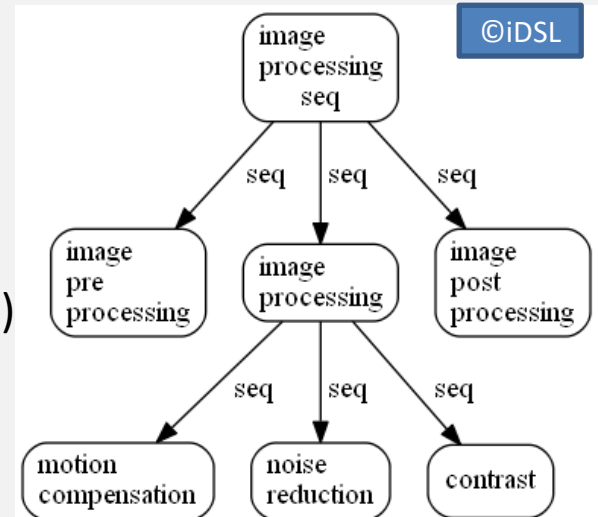
©iDSL

LANGUAGE CONSTRUCTS

PROCESS, RESOURCE, SYSTEM, SCENARIO, MEASURE AND STUDY

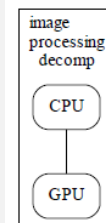
Section Process

```
ProcessModel image_processing_application seq {  
  atom image_pre_processing load 25  
  seq image_processing {  
    atom motion_compensation load 22  
    atom noise_reduction load uniform (40 70)  
    atom contrast load 67  
  }  
  atom image_post_processing load 5  
} // also supported: par, alt, palt, abstract and mutex
```



Section Resource

```
ResourceModel image_processing_PC decomp  
{ atom CPU rate 1 atom GPU rate 1 } connections { (CPU,GPU) }
```



LANGUAGE CONSTRUCTS

PROCESS, RESOURCE, SYSTEM, SCENARIO, MEASURE AND STUDY

Section System

Service image_processing_service

Process

image_processing_application

Resource

image_processing_PC

Mapping assign {

(image_pre_processing, CPU)

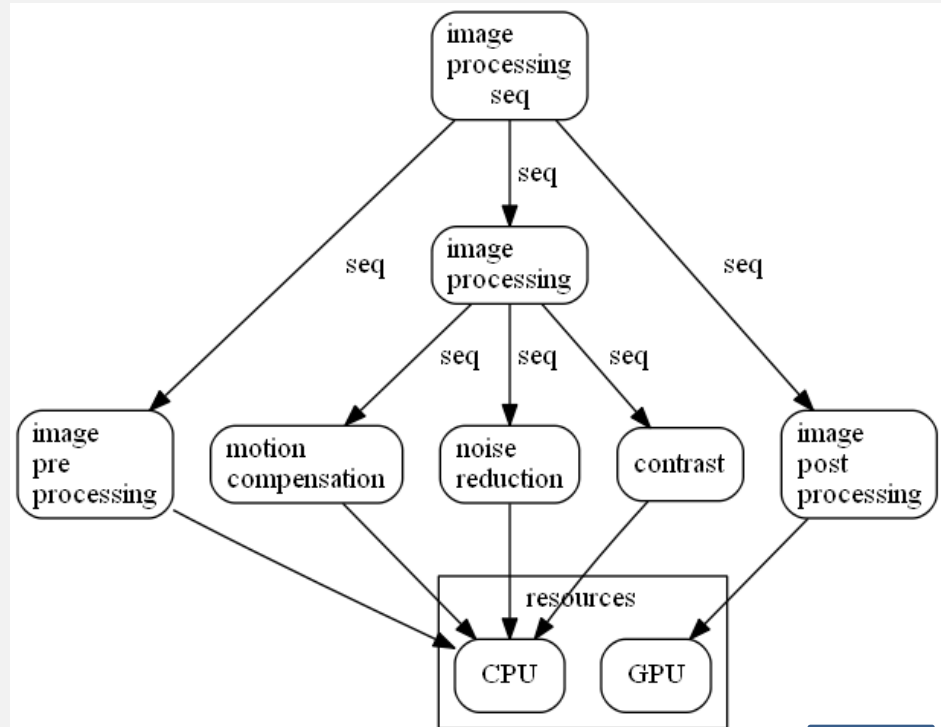
(motion_compensation, CPU)

(noise_reduction, CPU)

(contrast, CPU)

(image_post_processing, GPU)

}



©iDSL

LANGUAGE CONSTRUCTS

PROCESS, RESOURCE, SYSTEM, SCENARIO, MEASURE AND STUDY

Section Scenario

Scenario image_processing_run

ServiceRequest image_processing_service **at time** 0, 400, ...

ServiceRequest image_processing_service **at time** 0, 400, ...

LANGUAGE CONSTRUCTS

PROCESS, RESOURCE, SYSTEM, SCENARIO, MEASURE AND STUDY

Section Scenario

Scenario image_processing_run

ServiceRequest image_processing_service **at time** 0, 400, ...

ServiceRequest image_processing_service **at time** 0, 400, ...

Section Measure

Measure ServiceResponse times using 1 runs of 280 **ServiceRequests**

Measure theoretical bounds

LANGUAGE CONSTRUCTS

PROCESS, RESOURCE, SYSTEM, SCENARIO, MEASURE AND STUDY

Section Scenario

Scenario image_processing_run

ServiceRequest image_processing_service **at time** 0, 400, ...

ServiceRequest image_processing_service **at time** 0, 400, ...

Section Measure

Measure **ServiceResponse** times using 1 runs of 280 **ServiceRequests**

Measure theoretical bounds

Section Study

Scenario image_processing_run

LANGUAGE CONSTRUCTS

PROCESS, RESOURCE, SYSTEM, SCENARIO, MEASURE AND STUDY

Section Scenario

Scenario image_processing_run

ServiceRequest image_processing_service **at time** 0, 400, ...

ServiceRequest image_processing_service **at time**

dspace(starting_offset), (400+dspace(starting_offset)), ...

Section Measure

Measure ServiceResponse times using 1 runs of 280 **ServiceRequests**

Measure theoretical bounds

Section Study

Scenario image_processing_run

DesignSpace (starting_offset { "0" "20" "40" "60" "80" "120" "160" "200" })

OVERVIEW

- Introduction
- **iDSL tool and solution chain**
- iDSL language constructs
- **iDSL performance evaluation**
 - **Way of working**
 - MODES simulations
 - UPPAAL model checking
- Conclusion

IDSL PERFORMANCE EVALUATION

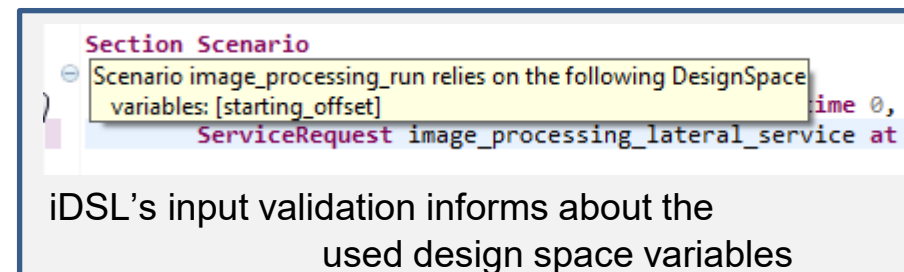
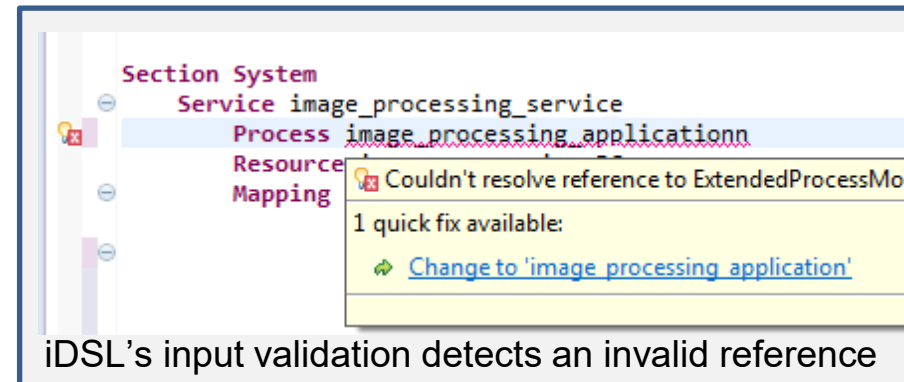
WAY OF WORKING

Assumption: iDSL is properly installed on your computer

- **Modelling**
 - Specify the model using iDSL language
 - Input completion and validation

- **Computation**
 - Sit back and relax

- **Analyzing**
 - Browse through the graphs, charts and numbers

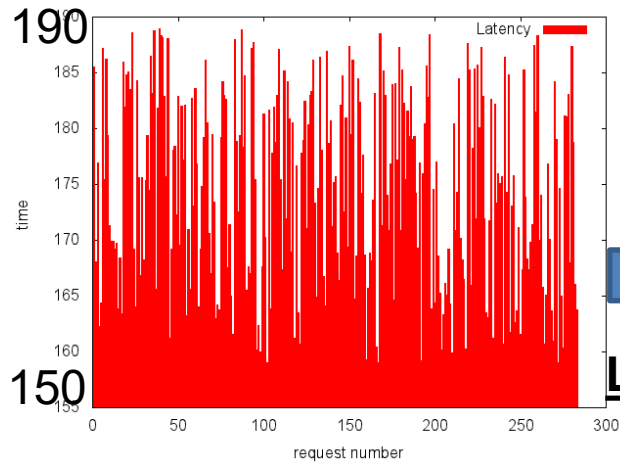
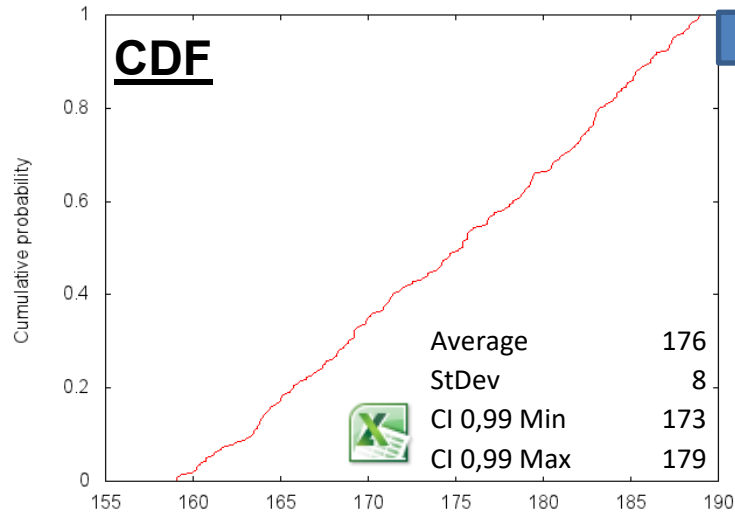


OVERVIEW

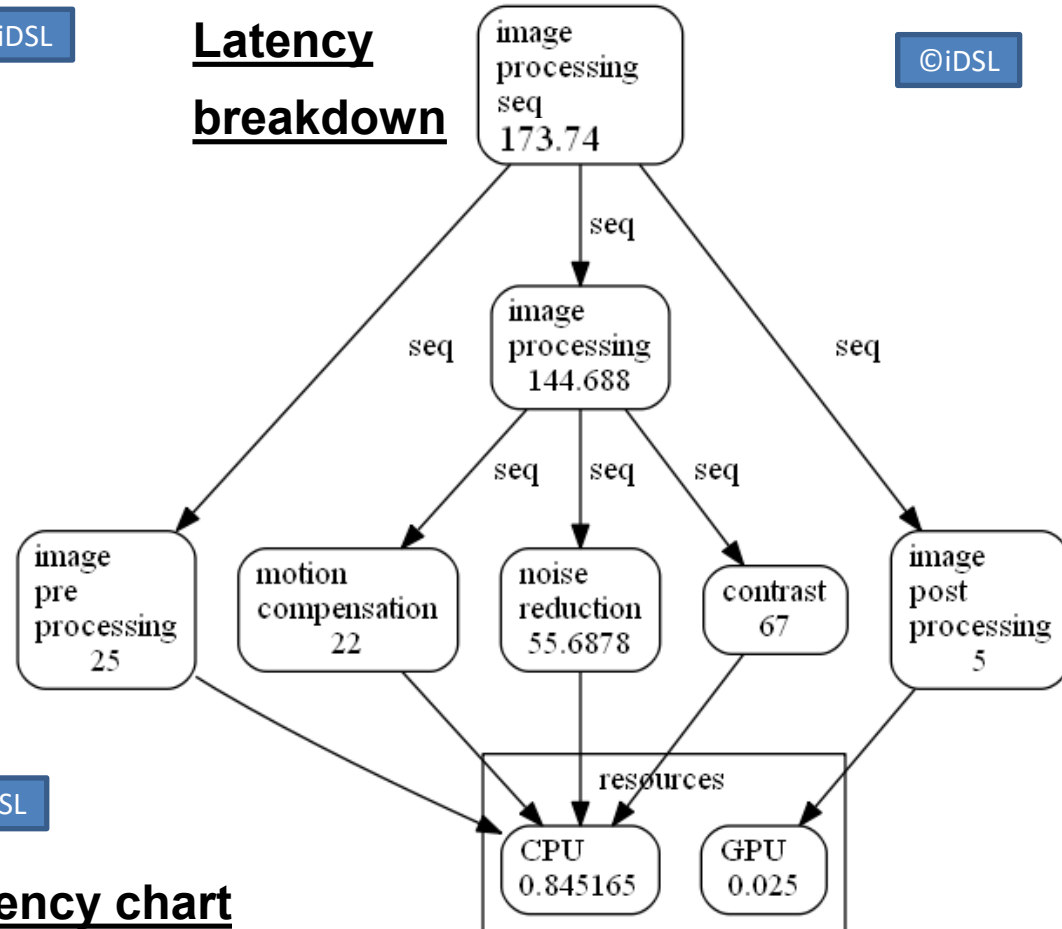
- Introduction
- About **iDSL**
 - solution chain
 - language constructs
- **iDSL performance evaluation**
 - Way of working
 - **MODES simulations**
 - UPPAAL model checking
- Conclusion

IDSL PERFORMANCE EVALUATION

MODES SIMULATIONS: OFFSET 200

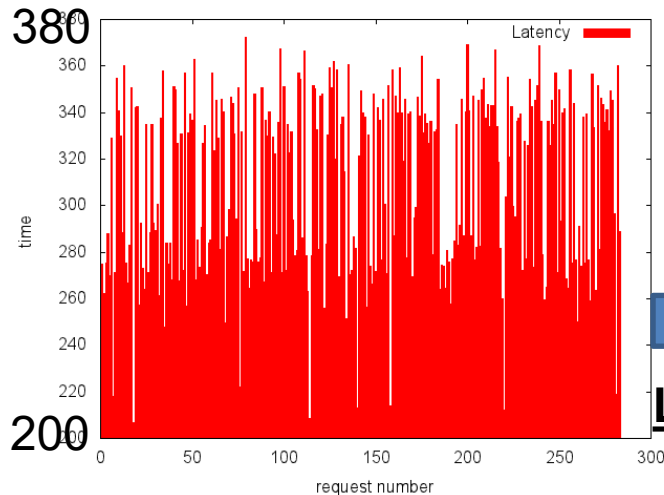
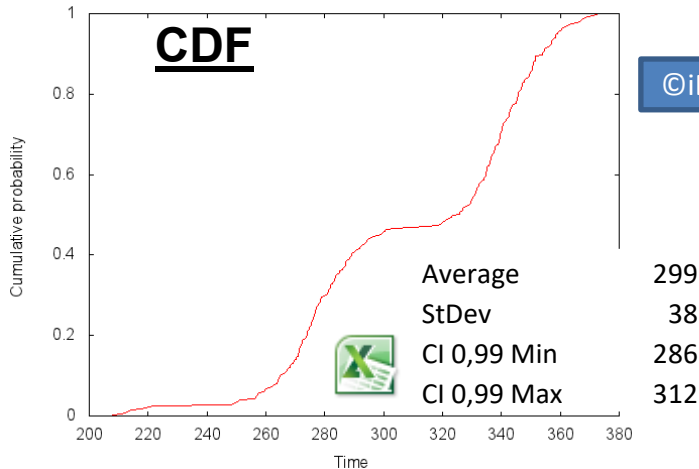


Latency breakdown

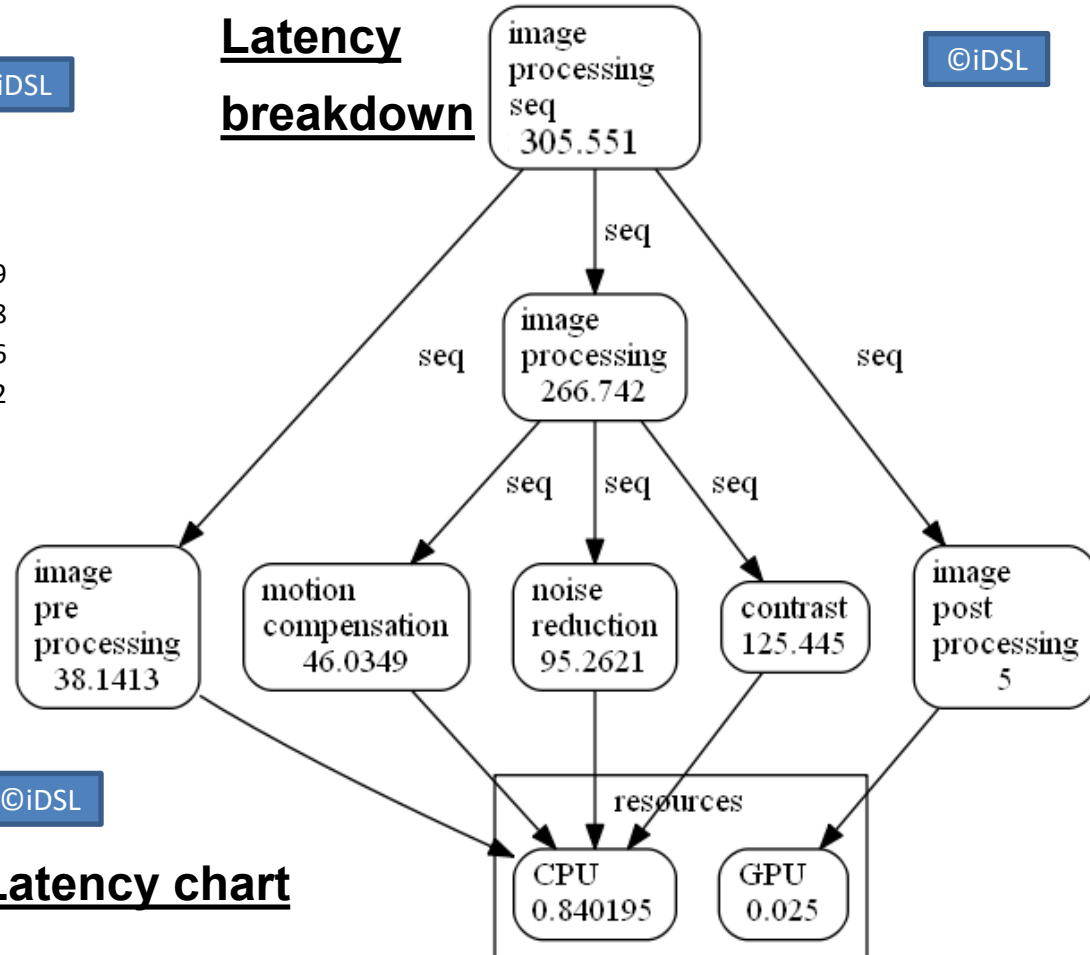


IDSL PERFORMANCE EVALUATION

MODES SIMULATIONS: OFFSET 0



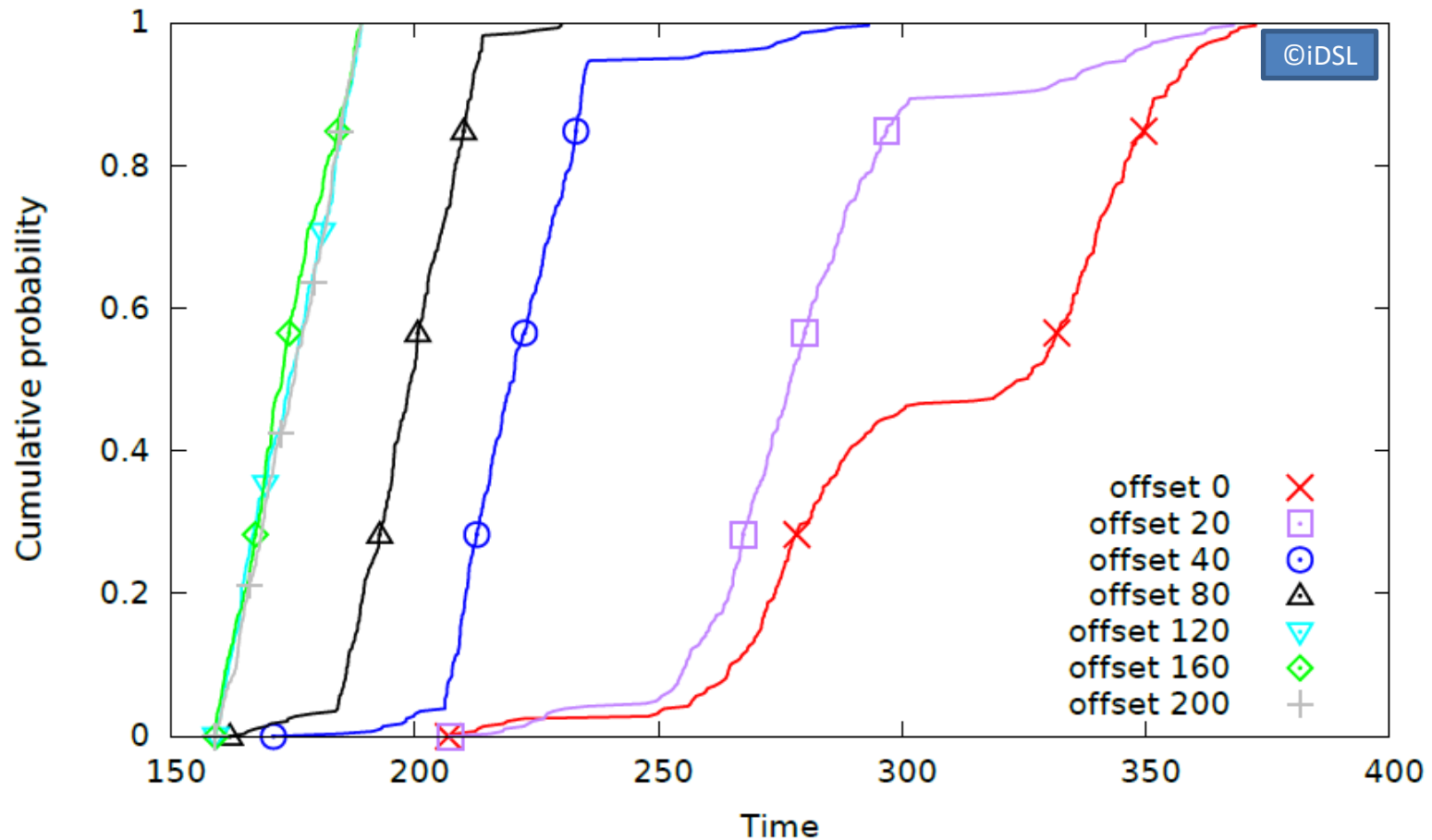
Latency breakdown



IDSL PERFORMANCE EVALUATION

MODES SIMULATIONS: CDF WITH ALL DESIGN INSTANCES

Fully automated: visualizing **multiple** services or design space instances



OVERVIEW

- Introduction
- About **iDSL**
 - solution chain
 - language constructs
- **iDSL performance evaluation**
 - Way of working
 - MODES simulations
 - **UPPAAL model checking**
- Conclusion

IDSL PERFORMANCE EVALUATION

UPPAAL MODEL CHECKING

- Underlying model
 - **UPPAAL** uses Timed Automata (**TA**)
 - **MODES** uses Stochastic Timed Automata (**STA**)
- We'd like to take advantage of the **expressiveness** of STAs
- Therefore, we create TAs based on STAs, for UPPAAL:
 - Real numbers → 16 bits integers
 - Probabilistic choice → non-deterministic choice
 - (In)finite distributions → non-deterministic choice
 - Removal of non-functional variables to reduce **the state space**

IDSL PERFORMANCE EVALUATION

UPPAAL MODEL CHECKING: DEFINITION

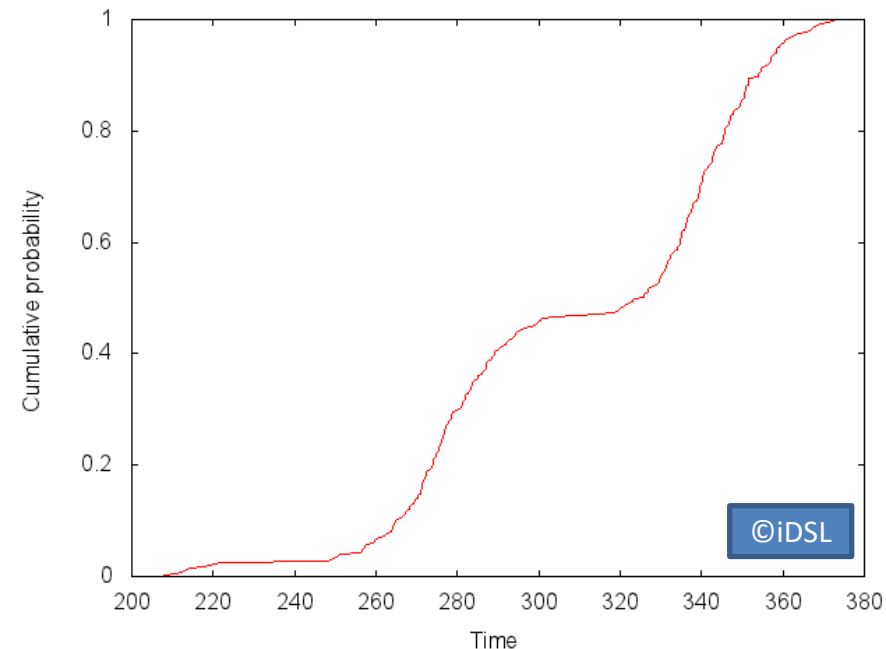
- What are **absolute bounds**?
 - Valid**: they do not intersect the values
 - Strict**: they span the smallest range

$$\begin{aligned} V1: Lb_f = v &\rightarrow CDF_f(v - \epsilon) = 0 \\ V2: Ub_f = v &\rightarrow CDF_f(v) = 1 \\ S1: Lb_f = v &\rightarrow CDF_f(v) > 0 \\ S2: Ub_f = v &\rightarrow CDF_f(v - \epsilon) < 1 \end{aligned}$$

- UPPAAL's interface**



- Perform a binary search
 - complexity **$O(\log(n))$**
 - iterative UPPAAL calls



IDSL PERFORMANCE EVALUATION

UPPAAL MODEL CHECKING: ALGORITHM

- An automated binary search for **absolute bounds** using MCTAU/UPPAAL

```
LB (lbound,ubound){  
    if (ubound==lbound) return lbound  
    check_value=(lbound+ubound)/2  
    UPPAAL (p = probability(latency<check_value))  
  
    if ( p=0 ) LB (check_value,ubound)  
    else      LB (lbound,check_value) }
```

IDSL PERFORMANCE EVALUATION

UPPAAL MODEL CHECKING: ALGORITHM

- An automated binary search for **absolute bounds** using MCTAU/UPPAAL

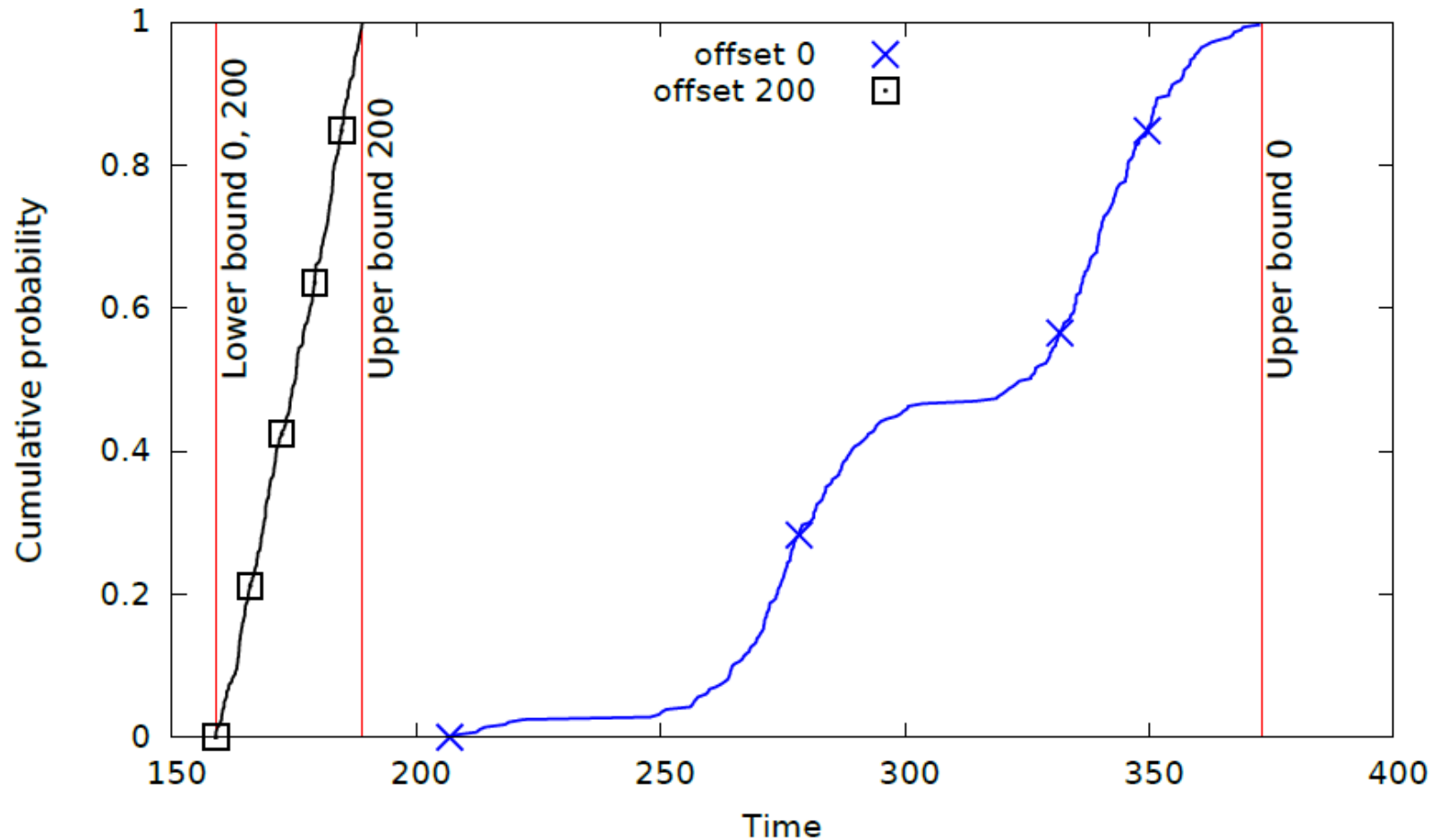
```
LB (lbound,ubound){  
    if (ubound==lbound) return lbound  
    check_value=(lbound+ubound)/2  
    UPPAAL (p = probability(latency<check_value))  
  
    if ( p=0 ) LB (check_value,ubound)  
    else      LB (lbound,check_value) }
```

An example run:

```
LB(0,1024)  -> LB(0,512)    -> LB(0,256) ->  
LB(128,256) -> LB(128,192) -> LB(128,160) ->  
LB(144,160) -> LB(152,160) -> LB(156,160) ->  
LB(158,160) -> LB(159,160) -> LB(159,159) -> 159
```

IDSL PERFORMANCE EVALUATION

UPPAAL MODEL CHECKING



OVERVIEW

- Introduction
- About **iDSL**
 - solution chain
 - language constructs
- iDSL performance evaluation
 - Way of working
 - MODES simulations
 - UPPAAL model checking
- **Conclusion**

CONCLUSION

- Our **industrial partner** Philips
 - designs **iXR machines** that are Medical Imaging Systems (MIS)
 - would like to **predict performance** in addition to measuring
- We have constructed **iDSL** for this, a language for MIS, which
 - transforms into **MoDeST** for simulations and model checking
 - **visualizes** its results
- Our following steps are
 - Apply iDSL on an **extensive use case**
 - Extend iDSL with **probablistic model checking** via PRISM
 - Equip iDSL with automated **Design Space Exploration**